

Radix Tech Assessment – Senior Developer

Technical Assessment: Loan Payment System (Single Application)

Overview

You are tasked with building a **Loan Payment System** within a **single Spring Boot application**. The application should have two core domains:

1. **Loan Domain:** Manages loan creation and retrieval.
2. **Payment Domain:** Handles payments made towards loans.

The system should expose **REST APIs** for both domains, with each domain logically separated within the application. The application will use an **in-memory H2 database** for persistence.

Requirements

1. **Loan Domain:**
 - Implement the following API endpoint:
 - **POST /loans:** Creates a new loan with a specified loan amount and term.
 - **GET /loans/{loanId}:** Retrieves the details of a loan by its ID (loan amount, remaining balance, etc.).
2. **Payment Domain:**
 - Implement the following API endpoint:
 - **POST /payments:** Records a payment made towards a loan by providing the loanId and payment amount.
3. **Business Logic:**
 - A loan must have an initial amount and term.
 - Payments should reduce the loan balance. If the payment exceeds the remaining balance, raise an error.
 - If a loan is settled in full, moved it to a **SETTLED** state.
4. **Separation Across Domains:**
 - The Loan and Payment domains should be logically separated. For example, you can separate them into different packages, but you are free to structure your application as you see fit, as long as the separation of concerns is clear.
5. **Database**
 - The application should use an **H2 in-memory database** for persistence.
6. **Unit Testing:**
 - Write unit tests to ensure that:
 - Loans are created successfully.
 - Payments are processed and correctly reduce the loan balance.
7. **Documentation:**

- Provide a **README.md** file that includes:
 - Instructions on how to build and run the application.
 - How to test the APIs (using Postman, curl, or similar tools).
 - Any relevant setup or configuration instructions.
-

Instructions

1. Set Up the Project

- **Create a GitHub repository** for your solution.
- The repository should include the following:
 - **Spring Boot application** with the two domains (Loan and Payment).
 - **Unit tests** for the key functionality.

2. Implement the Loan Domain

- **Loan Entity:** Create an entity for Loan with the following fields:
 - loanId (unique identifier for the loan)
 - loanAmount (the total amount of the loan)
 - term (the duration of the loan in months)
 - status (ACTIVE or SETTLED, depending on whether the loan has been repaid in full)
- **Loan Controller:** Implement a REST controller with the following endpoints:
 - POST /loans: To create a new loan.
 - GET /loans/{loanId}: To retrieve loan details.

3. Implement the Payment Domain

- **Payment Entity:** Create an entity for Payment with the following fields:
 - paymentId (unique identifier for the payment)
 - loanId (the loan associated with the payment)
 - paymentAmount (the amount paid towards the loan)
- **Payment Controller:** Implement a REST controller with the following endpoint:
 - POST /payments: To record a payment for a loan.

4. Database

- The application should use an **H2 in-memory database** for persistence.

5. Unit Testing

- Write **unit tests** to verify:
 - A loan is successfully created and stored in the database.
 - A payment is successfully recorded and reduces the loan balance.
 - Overpayment results in an error being raised.
 - If a loan is paid in full, it moves to a SETTLED state.

6. Documentation

- Update the README.md file with instructions on
 - How to build and run the application.
 - How to test the APIs (using Postman or curl).
 - Any additional configuration or setup notes.

Submission

- **Create a new GitHub repository** for your solution and **commit the code** there.
- Include the necessary documentation and instructions in the **README.md** file.
- **Share the link** to your GitHub repository with us once completed.

Good luck, and we look forward to reviewing your solution!
